R E V E R S E E N G I N E E R I N G

Amazon A10 Client-Side Behavioral Scoring System

RXVM Bytecode VM • Neural Network Model • Behavioral Telemetry

March 2026

Verified across multiple Amazon search result pages

Executive Summary

Amazon embeds an obfuscated virtual machine (RXVM) into every search results page. This VM silently collects 31 behavioral signals from each visitor — mouse movement patterns, scroll behavior, click timing, viewport dimensions, and behavioral hash distributions — then feeds them through a client-side neural network that produces a real-time “humanness” score between 0 and 1.

This report documents the complete reverse engineering of this system: the bytecode VM architecture, the 15 embedded payloads, the neural network model (architecture, weights, and normalization parameters), and the feature collection pipeline. All findings were verified by disassembling bytecode extracted from live Amazon pages in March 2026.

System Overview

How It Works

When a user loads an Amazon search results page, the following sequence executes invisibly:

- 15 base64-encoded RXVM bytecode payloads are decoded and executed in a custom virtual machine
- Event listeners attach to mousemove, mousedown/mouseup, scroll, and click events
- Behavioral data is accumulated into statistical aggregates (sum, mean, count, standard deviation)
- 31 features are normalized using pre-trained population statistics (mean, std, clip bounds)
- Features pass through a 3-layer neural network: Dense(31→16) → BatchNorm → ReLU → Dense(16→16) → BatchNorm → ReLU → Dense(16→1) → Sigmoid
- Output score (0 = bot, 1 = human) is sent server-side alongside the user’s session and request IDs

Scope of Collection

This is not a simple cookie or fingerprint check. The system builds a real-time behavioral profile by continuously monitoring how the user physically interacts with the page. The neural network then classifies the interaction pattern, not the device or identity.

The RXVM Bytecode Virtual Machine

Amazon’s behavioral code is not delivered as readable JavaScript. Instead, it ships as base64-encoded bytecode executed by a custom virtual machine embedded in the page. This VM (designated RXVM from its “RX” magic bytes) implements a complete instruction set with arithmetic, control flow, closures, property access, and iteration.

VM Characteristics

Property	Detail

Magic Bytes	`0x52 0x58 ("RX")`
Version	`1`
Encoding	`XOR cipher, key = (offset % 127) + 37`
Integers	`Variable-length (varint), 16-bit signed, 32-bit signed`
Floats	`IEEE 754 float32, plus custom 8-bit minifloat`
Call Convention	`Accumulator-based with local/global slots`
Total Payloads	`15 (identical across all search pages)`
Total Bytecode	`~13 KB compressed`

Instruction Set

The VM supports 30+ opcodes covering storage (STORE_LOCAL, STORE_GLOBAL), evaluation (EVAL, PROP_SET), control flow (JUMP, JMP_TRUE, JMP_FALSE), function calls (CALL_0 through CALL_N, NEW_0 through NEW_N), closures (MAKE_CLOSURE, MAKE_NAMED_CLOSURE), iteration (ITER_INIT, ITER_NEXT), and binary operations (arithmetic, bitwise, comparison, logical). Notably, it includes a custom 8-bit minifloat format used for compact model weight storage.

Payload Map

Each of the 15 payloads serves a specific function in the data collection and scoring pipeline:

#	Bytes	Function
0	1344	Core VM bootstrap, crypto init, hash functions (imul, MurmurHash3)
1	871	Statistical aggregation: sum, average, std deviation, float16 encode/decode
2	194	Fingerprint generator (fnp/fnpb/fnpv)
3	717	Data collection orchestrator, localStorage persistence, feature serialization
4	845	Environment probes: webdriver detection, navigator/document inspection
5	615	Mouse click timing collector (mousedown/mouseup intervals)
6	936	Mouse movement collector (mousemove acceleration, velocity)
7	631	Scroll behavior collector (scroll position, frequency, direction)
8	360	Persistent event counter (localStorage rx-pnt)
9	319	Float16 behavioral summary encoder
10	1026	Neural network layer primitives: Dense, BatchNorm, ReLU, Sigmoid
11	2875	Model definition: architecture, weights, feature norms, inference entry
12	673	Click/move distance tracker (Euclidean distance, sqrt/pow)

13	304	DOM element tagging and UEX telemetry bridge
14	1193	CSS overlay detection (pplx-agent regex), cookie checks, anti-tamper

Key finding: Payload [14] contains regex patterns matching **`pplx-agent`** — Amazon is specifically detecting Perplexity AI's browser overlay/agent. This is direct evidence of targeted countermeasures against AI-assisted browsing.

Feature Collection Pipeline

The system collects 31 features organized into 7 categories. Each feature is normalized using population-level statistics (mean, standard deviation, clip bounds) derived from Amazon's training data — effectively calibrated against the behavior of hundreds of millions of real users.

Top 10 Features by Model Influence

#	Feature	Category	Description	Importance
1	**`ma_c`**	Mouse Movement	Movement segment count	0.623
2	**`sc_c`**	Scroll	Scroll event count	0.324
3	**`hdl_2`**	Behavioral Hash	Hash distribution level 2	0.254
4	**`hdl_8`**	Behavioral Hash	Hash distribution level 8	0.240
5	**`hdl_7`**	Behavioral Hash	Hash distribution level 7	0.219
6	**`sw`**	Viewport/Screen	Screen width (raw)	0.213
7	**`hdl_6`**	Behavioral Hash	Hash distribution level 6	0.184
8	**`ma_s`**	Mouse Movement	Acceleration sum	0.145
9	**`sc_s`**	Scroll	Total scroll displacement	0.131
10	**`mc_c`**	Click Patterns	Click interval count	0.104

Category Importance (Aggregated)

Category	Score	Relative Weight
Mouse Movement	0.461	
Scroll Behavior	0.290	
Behavioral Hash	0.206	
Viewport/Screen	0.155	
Click Patterns	0.112	
Page Structure	0.104	
Mouse Timing	0.083	

Key insight: Mouse movement (acceleration patterns, segment count) is the single most influential category, contributing 46.1% of the model's weighting. This makes sense — natural mouse acceleration is extremely difficult for bots to simulate convincingly, while scroll counts and click timing are easier to fake.

Neural Network Architecture

Model Structure

#	Layer	Shape	Parameters	Activation
1	**Dense (input)**	31 → 16	512	—
2	**BatchNorm**	16	64	—
3	**ReLU**	16	0	max(0, x)
4	**Dense (hidden)**	16 → 16	272	—
5	**BatchNorm**	16	64	—
6	**ReLU**	16	0	max(0, x)
7	**Dense (output)**	16 → 1	17	—
8	**Sigmoid**	1	0	$1/(1+e^{-x})$

Total trainable parameters: 929. This is an intentionally tiny model — small enough to run in real-time in the browser without perceptible latency, yet expressive enough to distinguish human behavior from automated scripts.

Output Layer Analysis

The final dense layer maps 16 hidden neurons to a single output. The sign and magnitude of each weight reveals which internal representations push toward "human" vs. "bot" classification:

Neuron	Weight	Direction	Strength
neuron[0]	+1.1250	↑ **Human**	+++++
neuron[1]	+1.3750	↑ **Human**	++++++
neuron[2]	-1.3750	↓ **Bot**	———
neuron[3]	+1.2500	↑ **Human**	+++++
neuron[4]	-0.9375	↓ **Bot**	——
neuron[5]	-1.5000	↓ **Bot**	———
neuron[6]	+1.8750	↑ **Human**	++++++++
neuron[7]	+1.1250	↑ **Human**	+++++
neuron[8]	-1.8750	↓ **Bot**	————

neuron[9]	-1.2500	↓ Bot	———
neuron[10]	-1.0000	↓ Bot	———
neuron[11]	+2.5000	↑ Human	++++++++++
neuron[12]	+1.0000	↑ Human	++++
neuron[13]	+2.5000	↑ Human	++++++++++
neuron[14]	+0.8750	↑ Human	++++
neuron[15]	+1.5000	↑ Human	++++++

Bias = −0.5526. The negative output bias means the model starts slightly skeptical — you have to earn your "human" score through positive behavioral signals. 10 of 16 neurons push toward "human" while 6 push toward "bot," suggesting the network encodes both positive human indicators and negative bot indicators as separate internal features.

Feature Normalization

Each of the 31 input features is normalized using 5 parameters derived from Amazon's training population: mean, standard deviation, clip maximum, clip minimum, and offset. This effectively calibrates the model against the behavioral distribution of Amazon's actual user base. The normalization formula is:

$$\hat{x} = (\text{clamp}(x, \text{min}, \text{max}) - \text{offset} - \text{mean}) / \text{std}$$

Selected normalization parameters revealing population-level behavioral statistics:

Feature	Mean	Std Dev	Clip Max	Clip Min	Offset
sc_s	517.61	259.05	2000	0	0.00
ms_s	308.96	74.32	2000	0	0.00
ma_c	39.01	9.33	250	0	0.00
vh	158.49	763.62	5000	0	763.62
sw	623.43	900.96	5000	0	900.96
sl	7.41	13.57	20	0	0.00

Anti-Tamper and Detection Targets

WebDriver Detection

Payload [4] directly checks **`navigator.webdriver`** — a property set to `true` by automation frameworks like Selenium, Puppeteer, and Playwright. The payload also enumerates `Object.keys()` on the navigator and document objects, checking for properties injected by automation tools.

AI Browser Agent Detection

Payload [14] contains compiled regex patterns specifically targeting Perplexity AI's browser overlay:

```
^pplx-agent(?:-[0-9]+_[0-9]+)?-overlay(?:-(?:base|-stop-butt...)
```

This scans CSS rules (via document.styleSheets) for class names matching Perplexity's injected overlay elements. The system also checks cookies (amzn-ctxv-id, amzn-css-id) and runs periodic interval-based rescans, indicating active monitoring throughout the session rather than a one-time check.

localStorage Persistence

Payload [8] maintains a persistent event counter under the key **`rx-pnt`** in localStorage. This means Amazon can correlate behavioral patterns across page loads within the same browser session, building a longitudinal profile even without server-side cookies.

Privacy Implications

This system raises significant questions about informed consent and proportionality:

- **Invisible execution:** The bytecode VM runs silently with no user-visible indication. The XOR-encoded bytecode format is specifically designed to resist casual inspection.
- **Continuous monitoring:** Event listeners capture every mouse movement, click, and scroll event for the duration of the page visit. This is granular physical behavior tracking.
- **Population-calibrated:** The normalization parameters reveal that this model was trained on data from Amazon's actual user population, meaning real users' behavioral data was collected to build the training set.
- **Cross-session persistence:** localStorage-based counters allow behavioral fingerprinting across multiple page loads.
- **No disclosure:** None of this behavioral analysis is mentioned in any user-facing interface element on the search results page.

Methodology

This analysis was conducted using the following approach:

- **Bytecode extraction:** Raw HTML from Amazon search result pages was saved and regex-scanned for rx.ex64("...") base64 payloads.
- **Disassembly:** A custom Python disassembler (rxvm_disasm.py) was developed by reverse-engineering the VM's instruction dispatch, XOR key schedule, varint encoding, and value type system.
- **Model extraction:** Neural network weights, biases, batch normalization parameters, and feature norms were extracted from the disassembled bytecode and exported to JSON.
- **Verification:** Multiple Amazon search pages ("hat", "dodge dakota") were tested in March 2026, producing byte-identical disassembly output across all pages, confirming the static nature of the bytecode payloads.

- **Independent inference:** A standalone Python implementation (a10_model.py) reproduces the full forward pass, validating the extracted model against expected behavior.

Conclusions

Amazon's A10 client-side behavioral scoring system represents one of the most sophisticated and widely-deployed invisible ML inference systems on the consumer web. It combines a custom bytecode VM for obfuscation, continuous behavioral telemetry, and a compact neural network into a system that runs entirely client-side, producing real-time bot scores with no server round-trip required.

The complete extraction of this system — VM instruction set, payload functions, model architecture, trained weights, and normalization parameters — demonstrates that security-through-obscurity approaches to client-side ML are fundamentally limited. Any model that runs in the browser can, with sufficient effort, be fully reverse-engineered.

The 31-feature behavioral model reveals what Amazon considers the strongest signals of human vs. automated interaction: mouse acceleration patterns dominate, followed by scroll behavior and behavioral hash distributions. This priority ordering provides insight into which behavioral dimensions are hardest for current bot technology to convincingly simulate.